

Playwright Complete Guide

Playwright Complete Guide

Table of Contents

PART I — FOUNDATIONS

1. What Is Playwright & Why Use It
2. Playwright vs Selenium / Cypress / Puppeteer
3. Environment Setup & Configuration
4. Architecture: Browser, Context, Page

PART II — CORE CONCEPTS

5. Locators & Selector Strategy
6. Auto-Waiting & Wait Strategies
7. Actions & Interactions
8. Assertions & Validations
9. Page Object Model & Component Object Model

PART III — HANDLING REAL APPLICATION BEHAVIOR

10. Authentication & Session State
11. File Uploads & Downloads
12. Dialogs (Alerts, Prompts, Confirms)
13. Frames & iFrames
14. Popups, New Tabs & Multiple Contexts
15. Network Interception & Mocking
16.  Cookies, LocalStorage & Browser Storage
17.  Shadow DOM
18.  Mobile & Device Emulation

PART IV — TESTING STRATEGY & TYPES

19. API Testing (REST, GraphQL, Hybrid)
20. Visual Regression Testing
21. Accessibility Testing
22. Performance Testing
23.  Component Testing (Playwright CT)

PART V — ENTERPRISE ENGINEERING PRACTICES

24. Test Organization & Structure
25. Custom Fixtures
26. Data-Driven & Tagged Tests
27. CI/CD Integration & Test Sharding
28.  Test Reporting (HTML, JUnit, Allure)
29. Debugging & Troubleshooting
30. Advanced Resilience & Self-Healing Patterns
31.  Codegen & the Trace Recorder
32.  Environment & Secrets Management

PART VI — THE FUTURE

33. AI-Driven Testing

PART VII — INTERVIEW PREPARATION

34. Scenario-Based Questions (Deep Dives)
35. Best Practices & Common Pitfalls Cheat Sheet

Playwright Complete Guide

Consolidated Handbook + Interview Q&A Reference

Sources merged: “Playwright Automation with TypeScript — Enterprise Guide” and “Playwright Automation Interview Questions.” Overlapping content has been unified into single explanations; unique material from each has been preserved; gaps have been filled with additional topics (marked **NEW**).

Table of Contents

PART I — Foundations 1. What Is Playwright & Why Use It 2. Playwright vs Selenium / Cypress / Puppeteer 3. Environment Setup & Configuration 4. Architecture: Browser, Context, Page

PART II — Core Concepts 5. Locators & Selector Strategy 6. Auto-Waiting & Wait Strategies 7. Actions & Interactions 8. Assertions & Validations 9. Page Object Model & Component Object Model

PART III — Handling Real Application Behavior 10. Authentication & Session State 11. File Uploads & Downloads 12. Dialogs (Alerts, Prompts, Confirms) 13. Frames & iFrames 14. Popups, New Tabs & Multiple Contexts 15. Network Interception & Mocking 16. **NEW** Cookies, LocalStorage & Browser Storage 17. **NEW** Shadow DOM 18. **NEW** Mobile & Device Emulation

PART IV — Testing Strategy & Types 19. API Testing (REST, GraphQL, Hybrid UI+API) 20. Visual Regression Testing 21. Accessibility Testing 22. Performance Testing 23. **NEW** Component Testing (Playwright CT)

PART V — Enterprise Engineering Practices 24. Test Organization & Structure 25. Custom Fixtures 26. Data-Driven & Tagged Tests 27. CI/CD Integration & Test Sharding 28. **NEW** Test Reporting (HTML, JUnit, Allure) 29. Debugging & Troubleshooting 30. Advanced Resilience & Self-Healing Patterns 31. **NEW** Codegen & the Trace Recorder 32. **NEW** Environment & Secrets Management

PART VI — The Future 33. AI-Driven Testing

PART VII — Interview Preparation 34. Scenario-Based Questions (unique deep-dives) 35. Best Practices & Common Pitfalls Cheat Sheet

PART I — FOUNDATIONS

1. What Is Playwright & Why Use It

Playwright is Microsoft's open-source end-to-end testing framework, built by the same team that created Puppeteer at Google, released in 2020. It drives Chromium, Firefox, and WebKit through a single API, with bundled browser binaries so behavior is consistent across machines ("no works-on-my-machine issues").

Key advantages: - True cross-browser support, including WebKit/Safari (desktop and iOS) - Built-in auto-wait — eliminates most explicit waits and flaky-test causes - Multiple selector strategies, prioritizing user-facing attributes over CSS/XPath - Native network interception and request mocking - Mobile emulation and multi-tab/multi-context support - Parallel execution and test sharding out of the box - Built-in screenshots, video, and trace recording

The Testing Trophy (vs. the Testing Pyramid)

The traditional Testing Pyramid (70% unit / 20% integration / 10% E2E) was designed for an era when E2E tests were slow and unreliable. Playwright's speed and reliability make the **Testing Trophy** a better model:

- 10% Static Analysis (TypeScript, ESLint)
- 20% Unit Tests (pure business logic)
- 50% Integration Tests (components + API together)
- 20% E2E Tests (critical user journeys)

Enterprise insight: A single well-designed E2E test can validate frontend, backend, database, and API contracts together — one banking login test can confirm UI rendering, form validation, auth API, database lookup, session creation, and routing in one pass.

```
test('complete banking login flow', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('Account Number').fill('123456789');
  await page.getByLabel('PIN').fill('1234');
  await page.getByRole('button', { name: 'Login' }).click();

  await expect(page).toHaveURL('/dashboard');
  await expect(page.getByText('Account Balance')).toBeVisible();
});
```

2. Playwright vs Selenium / Cypress / Puppeteer

Comparison	Playwright's edge
vs Selenium	20-50% faster; auto-downloads browsers; modern async/await API; built-in auto-wait (Selenium needs explicit <code>WebDriverWait</code>)
vs Cypress	Native WebKit/Safari support; true multi-tab support; native mobile browser emulation; multi-language bindings (JS/TS, Python, .NET, Java)
vs Puppeteer	Adds Firefox + WebKit (Puppeteer is Chromium-only); purpose-built for testing; user-facing locators instead of raw CSS/XPath

3. Environment Setup & Configuration

Prerequisites: Node.js 16+ (LTS), VS Code, basic TypeScript.

```
mkdir playwright-automation && cd playwright-automation
npm init -y
npm init playwright@latest
```

Answer the setup prompts: TypeScript → Yes, test directory → `tests`, GitHub Actions → Yes, install browsers → Yes.

Project structure:

```
playwright-automation/
├─ tests/
│   └─ example.spec.ts
├─ playwright.config.ts
└─ package.json
```

Base configuration:

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  timeout: 30 * 1000,
  fullyParallel: true,
  retries: process.env.CI ? 2 : 0,
  reporter: 'html',
  use: {
    baseURL: 'http://localhost:3000',
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
  },
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox', use: { ...devices['Desktop Firefox'] } },
    { name: 'webkit', use: { ...devices['Desktop Safari'] } },
  ],
});
```

Verify with `npx playwright test`. Use the VS Code Playwright extension for the test runner UI and visual locator picker.

4. Architecture: Browser, Context, Page

Test code runs in Node.js; Playwright drives browsers via the DevTools/CDP-style protocol; browsers run as separate processes.

- **Browser** — the actual browser process: `chromium.launch({ headless: false })`
- **Context** — an isolated session (own cookies, cache, storage). No data is shared between contexts.
- **Page** — a single tab within a context.

```
const browser = await chromium.launch();
const context1 = await browser.newContext();
const context2 = await browser.newContext();
const page1 = await context1.newPage();
const page2 = await context2.newPage();
```

Use cases for multiple contexts: simulating different logged-in users, testing permissions, isolating test data, running scenarios in parallel within one browser process.

PART II — CORE CONCEPTS

5. Locators & Selector Strategy

Playwright's `Locator` API is lazy (resolves at the time of the action), auto-waits, and is strict by default (fails on ambiguous multi-matches) — this is why it has replaced the older `page.$()` / `page.$$()` `ElementHandle` API, which resolves immediately, doesn't auto-wait, and can go stale as the DOM changes.

Priority order (best → last resort):

```
// 1. Role-based (BEST – reflects how users/assistive tech perceive the page)
page.getByRole('button', { name: 'Submit' });
page.getByRole('textbox', { name: 'Username' });

// 2. Label-based (great for forms)
page.getByLabel('Email address');

// 3. Text-based
page.getByText('Welcome');
page.getByText(/welcome/i);

// 4. Placeholder / Alt text / Title
page.getByPlaceholder('Enter your email');
page.getByAltText('Logo');
page.getByTitle('Close dialog');

// 5. Test ID (stable, but not user-facing)
page.getByTestId('submit-btn');

// 6. CSS/XPath (avoid – brittle, tied to implementation)
page.locator('#element-id');
```

Chaining and filtering:

```
await page.locator('.user-section').getByRole('button', { name: 'Edit' }).click();

await page.getByRole('listitem').filter({ hasText: 'Product 1' }).click();
```

Multiple elements:

```
const count = await page.getByRole('listitem').count();
await page.getByRole('button').first().click();
await page.getByRole('button').nth(2).click();
```

Best practice: Prefer `getByRole()` / `getByLabel()` over CSS selectors and test IDs — they're the most resilient to markup changes and they double-check your app's accessibility.

6. Auto-Waiting & Wait Strategies

Before every action, Playwright automatically waits for the element to be: - Attached to the DOM - Visible - Stable (not mid-animation) - Receiving events (not obscured by another element) - Enabled

```
// No explicit wait needed – Playwright waits for all five conditions above
await page.getByRole('button', { name: 'Submit' }).click();
```

Default timeout is 30 seconds; override per-action or globally:

```
await page.getByRole('button').click({ timeout: 5000 });
page.setDefaultTimeout(60000);
```

Explicit wait strategies for the cases auto-wait doesn't cover:

```
// Element state
await page.locator('#element').waitFor({ state: 'visible' }); // also: hidden, attached, detached

// Load states
await page.waitForLoadState('load');
await page.waitForLoadState('domcontentloaded');
await page.waitForLoadState('networkidle');

// Selector / URL / function / response / event
await page.waitForSelector('#element', { state: 'visible' });
await page.waitForURL(/.*profile.*/);
await page.waitForFunction(() => document.querySelectorAll('.item').length > 5);
await page.waitForResponse(res => res.url().includes('/api/') && res.status() === 200);
await page.waitForEvent('download');
```

⚠ **Avoid** `page.waitForTimeout()`. Arbitrary sleeps make tests both slower *and* more flaky. Auto-waiting removes roughly 80% of flaky-test causes on its own — trust it before reaching for a manual wait.

7. Actions & Interactions

```
// Clicking
await page.getByRole('button').click();
await page.getByText('File').dblclick();
await page.getByText('Item').click({ button: 'right' });
await page.getByText('Link').click({ modifiers: ['Control'] });

// Typing
await page.getByLabel('Username').fill('john.doe'); // clears first
await page.getByLabel('Search').type('playwright'); // doesn't clear
await page.keyboard.press('Enter');

// Forms
await page.getByRole('checkbox', { name: 'Terms' }).check();
await page.getByRole('radio', { name: 'Option 1' }).check();
await page.getByLabel('Country').selectOption('USA');
await page.getByLabel('Country').selectOption({ label: 'United States' });

// Mouse
await page.getByRole('link', { name: 'Products' }).hover();
await page.getByText('Drag').dragTo(page.getByText('Drop here'));
```

8. Assertions & Validations

Playwright's `expect()` assertions **auto-retry** until timeout (default 5s) — this is why they're preferred over reading a value once and asserting on it manually.

```
// Page
await expect(page).toHaveURL(/dashboard/);
await expect(page).toHaveTitle(/Dashboard/);

// Visibility / state
await expect(page.getByText('Welcome')).toBeVisible();
await expect(page.getByText('Loading')).toBeHidden();
await expect(page.getByRole('button')).toBeEnabled();
await expect(page.getByLabel('Terms')).toBeChecked();

// Text & values
await expect(page.getByRole('heading')).toHaveText('Welcome');
await expect(page.getByRole('paragraph')).toContainText('Playwright');
await expect(page.getByLabel('Email')).toHaveValue(/.*@example\.com/);

// Attributes & count
await expect(page.getByRole('button')).toHaveAttribute('type', 'submit');
await expect(page.getByRole('listitem')).toHaveCount(5);

// Custom timeout
await expect(page.getByText('Slow')).toBeVisible({ timeout: 30000 });
```

```
// ✗ Avoid – no auto-retry
const text = await page.locator('h1').textContent();
expect(text).toBe('Dashboard');

// ✔ Prefer – auto-retries
await expect(page.locator('h1')).toHaveText('Dashboard');
```

Tip: Use `expect.soft()` for non-critical checks that shouldn't abort the rest of the test: `await expect.soft(element).toBeVisible();`

9. Page Object Model & Component Object Model

Page Object (encapsulates one page's locators + actions):

```
export class LoginPage {
  readonly page: Page;
  readonly emailInput: Locator;
  readonly passwordInput: Locator;
  readonly submitButton: Locator;

  constructor(page: Page) {
    this.page = page;
    this.emailInput = page.getByLabel('Email');
    this.passwordInput = page.getByLabel('Password');
    this.submitButton = page.getByRole('button', { name: 'Log in' });
  }

  async goto() { await this.page.goto('/login'); }

  async login(email: string, password: string) {
    await this.emailInput.fill(email);
    await this.passwordInput.fill(password);
    await this.submitButton.click();
  }
}
```

```
test('login test', async ({ page }) => {
  const loginPage = new LoginPage(page);
  await loginPage.goto();
  await loginPage.login('user@example.com', 'pass123');
  await expect(page).toHaveURL('/dashboard');
});
```

Component Object Model (COM) — for shared UI pieces:

Enterprise insight: Modern apps are component-based. If a `Header` appears on 15 pages and changes, a pure Page Object approach means updating 15 classes. Component Objects fix this — update `HeaderComponent` once, and every page that composes it is updated automatically. This can cut maintenance effort by roughly 80%.

```
export class HeaderComponent {
  readonly page: Page;
  readonly searchInput: Locator;
  readonly logoutBtn: Locator;

  constructor(page: Page) {
    this.page = page;
    this.searchInput = page.getByTestId('search');
    this.logoutBtn = page.getByTestId('logout');
  }

  async search(query: string) {
    await this.searchInput.fill(query);
    await this.page.keyboard.press('Enter');
  }
}

// Base page composes shared components
export class BasePage {
  readonly page: Page;
  readonly header: HeaderComponent;
  constructor(page: Page) {
    this.page = page;
    this.header = new HeaderComponent(page);
  }
}

export class DashboardPage extends BasePage {
  async navigateToTransactions() {
    await this.header.search('transactions');
  }
}
```

Best practice: Use COM for Header, Footer, Navigation, Modals — any UI that recurs across pages. Keep page objects focused (one page, one class), use `readonly` for locators, and extract a `BasePage` for shared functionality.

PART III — HANDLING REAL APPLICATION BEHAVIOR

10. Authentication & Session State

Saving login state once and reusing it across tests avoids repeating a 5-second UI login in every test.

```
// auth.setup.ts — runs once before the suite
import { test as setup } from '@playwright/test';
const authFile = 'playwright/.auth/user.json';

setup('authenticate', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('Email').fill('user@example.com');
  await page.getByLabel('Password').fill('SecurePass123');
  await page.getByRole('button', { name: 'Log in' }).click();
  await page.waitForURL('/dashboard');
  await page.context().storageState({ path: authFile });
});
```

```
// playwright.config.ts
projects: [
  { name: 'setup', testMatch: /\.*\setup\.ts/ },
  {
    name: 'chromium',
    use: { ...devices['Desktop Chrome'], storageState: 'playwright/.auth/user.json' },
    dependencies: ['setup'],
  },
],
```

Enterprise insight: Reusing storage state can save 5–10 seconds per test. Across 1,000 tests, that's 1.5–3 hours of CI time saved.

API-based authentication (skip the UI entirely for setup):

```
test.use({
  storageState: async ({}, use) => {
    const response = await fetch('https://api.example.com/login', {
      method: 'POST',
      body: JSON.stringify({ user: 'admin', pass: 'password' }),
    });
    const { token } = await response.json();
    await use({
      cookies: [],
      origins: [{ origin: 'https://example.com', localStorage: [{ name: 'authToken', value: token }] }],
    });
  },
});
```

The 80/20 Hybrid Rule (API setup + UI testing)

Enterprise insight: Use APIs for ~80% of test setup (creating users, seeding data, logging in) and reserve the UI for the ~20% you actually want to validate. API calls take ~200ms vs. a UI login's ~5s, they have no animation/spinner delays, and they let 1,000 tests run in minutes instead of hours.

```

test('purchase with API setup', async ({ request, page }) => {
  const userResponse = await request.post('/api/users', {
    data: { name: 'Test User', accountNumber: '123456789', balance: 10000 },
  });
  const user = await userResponse.json();

  const { token } = await (await request.post('/api/auth/login', {
    data: { accountNumber: '123456789', pin: '1234' },
  })).json();

  await page.context().addCookies([
    { name: 'auth_token', value: token, domain: 'localhost', path: '/' },
  ]);

  await page.goto('/shop');
  await page.getByRole('button', { name: 'Buy Now' }).click();
  await page.getByRole('button', { name: 'Confirm Purchase' }).click();
  await expect(page.getByText('Purchase Successful')).toBeVisible();
});

```

11. File Uploads & Downloads

Uploads:

```

await page.getByLabel('Upload').setInputFiles('file.pdf');
await page.getByLabel('Upload photos').setInputFiles(['photo1.jpg', 'photo2.jpg']);

// From an in-memory buffer (no file on disk needed)
await page.getByLabel('Upload').setInputFiles({
  name: 'report.pdf',
  mimeType: 'application/pdf',
  buffer: Buffer.from('file content'),
});

// Clear a file input
await page.getByLabel('Upload').setInputFiles([]);

// Native file-chooser dialog
const fileChooserPromise = page.waitForEvent('filechooser');
await page.locator('#upload-btn').click();
const fileChooser = await fileChooserPromise;
await fileChooser.setFiles('file.pdf');

```

Downloads:

```

const [download] = await Promise.all([
  page.waitForEvent('download'),
  page.getByText('Download Report').click(),
]);

await download.saveAs('/downloads/' + download.suggestedFilename());
expect(download.suggestedFilename()).toContain('report.pdf');

```

12. Dialogs (Alerts, Prompts, Confirms)

```

page.on('dialog', async dialog => {
  if (dialog.type() === 'prompt') {
    await dialog.accept('User Input');
  } else if (dialog.message().includes('Are you sure?')) {
    await dialog.accept();
  } else {
    await dialog.dismiss();
  }
});

// One-time handler
page.once('dialog', dialog => dialog.accept());
await page.locator('#delete-btn').click();

```

13. Frames & iFrames

```

const paymentFrame = page.frameLocator('iframe[name="payment"]');
await paymentFrame.getByLabel('Card Number').fill('4242424242424242');
await paymentFrame.getByLabel('Expiry').fill('12/25');
await paymentFrame.getByRole('button', { name: 'Pay' }).click();

// Other frame access patterns
const frame = page.frame({ url: /.frame\.html/ });
const frames = page.frames();
const mainFrame = page.mainFrame();

```

 **Domain note:** Payment gateways and embedded widgets frequently use iframes — this pattern is essential for banking and e-commerce test suites.

14. Popups, New Tabs & Multiple Contexts

```

const [popup] = await Promise.all([
  context.waitForEvent('page'),
  page.getByText('Open Terms').click(),
]);
await popup.waitForLoadState();
await popup.getByRole('button', { name: 'Accept' }).click();

```

Multi-user scenarios via separate contexts:

```

test('multi-user scenario', async ({ browser }) => {
  const adminContext = await browser.newContext({ storageState: 'admin-auth.json' });
  const adminPage = await adminContext.newPage();

  const userContext = await browser.newContext({ storageState: 'user-auth.json' });
  const userPage = await userContext.newPage();

  await adminPage.goto('/admin/resources');
  await adminPage.getByRole('button', { name: 'Create' }).click();

  await userPage.goto('/resources');
  await expect(userPage.getByText('New Resource')).toBeVisible();

  await adminContext.close();
  await userContext.close();
});

```

Enterprise insight: Multiple contexts are the standard way to test collaborative features, permission boundaries, and real-time multi-user workflows.

15. Network Interception & Mocking

```
// Mock a response
await page.route('**/api/users', route => {
  route.fulfill({
    status: 200,
    contentType: 'application/json',
    body: JSON.stringify([{ id: 1, name: 'Alice', role: 'Admin' }]),
  });
});

// Simulate an outage
await page.route('**/api/transactions', route => {
  route.fulfill({ status: 503, body: JSON.stringify({ error: 'Service temporarily unavailable' }) });
});

// Simulate slow network
await page.route('**/api/**', route => setTimeout(() => route.continue(), 5000));

// Modify outgoing requests
await page.route('**/api/**', async route => {
  const headers = route.request().headers();
  headers['Authorization'] = 'Bearer token123';
  await route.continue({ headers });
});

// Block resource types (e.g., images) for faster runs
await page.route('**/*.png,jpg,jpeg', route => route.abort());

// Observe traffic
page.on('request', r => console.log('→', r.method(), r.url()));
page.on('response', r => console.log('←', r.status(), r.url()));
await page.waitForResponse('**/api/users');
```

Enterprise insight: Deliberately simulating 503s, timeouts, and dropped connections is critical for banking-grade resilience testing — you’re verifying graceful degradation, not just the happy path.

16. Cookies, LocalStorage & Browser Storage

Not covered in either source document, but a common interview and real-world need.

```
// Read/set/clear cookies at the context level
const cookies = await context.cookies();
await context.addCookies([{ name: 'session', value: 'abc123', domain: 'example.com', path: '/' }]);
await context.clearCookies();

// LocalStorage / sessionStorage via page.evaluate
await page.evaluate(() => localStorage.setItem('theme', 'dark'));
const theme = await page.evaluate(() => localStorage.getItem('theme'));

// Seed storage before navigation using storageState origins
await context.addInitScript(() => {
  window.localStorage.setItem('featureFlag', 'true');
});
```

Tip: `storageState()` captures both cookies *and* localStorage/sessionStorage per origin — it’s the same mechanism used for auth-state reuse (Section 10), not just cookies.

17. Shadow DOM

Playwright pierces open shadow roots automatically with standard locators — no special API needed for open shadow DOM:

```
// Works even if #my-button lives inside a shadow root
await page.locator('#my-button').click();

// getByRole/getByText also traverse open shadow DOM
await page.getByRole('button', { name: 'Add to cart' }).click();
```

Closed shadow roots (rare, deliberately inaccessible) cannot be pierced by design — this is a browser limitation, not a Playwright one. If you control the component, prefer exposing test hooks (e.g., `data-testid`) at the light-DOM boundary instead of relying on closed shadow internals.

18. Mobile & Device Emulation

Referenced only in passing in the source material (“mobile emulation support”) — here’s the actual usage:

```
import { devices } from '@playwright/test';

// playwright.config.ts
projects: [
  { name: 'Mobile Chrome', use: { ...devices['Pixel 7'] } },
  { name: 'Mobile Safari', use: { ...devices['iPhone 14'] } },
],
```

```
// Ad-hoc in a test
const context = await browser.newContext({
  ...devices['iPhone 14'],
  geolocation: { latitude: 40.7128, longitude: -74.0060 },
  permissions: ['geolocation'],
});
```

```
test('touch gesture', async ({ page }) => {
  await page.touchscreen.tap(100, 200);
});
```

PART IV — TESTING STRATEGY & TYPES

19. API Testing (REST, GraphQL, Hybrid)

```
// GET
test('GET users', async ({ request }) => {
  const response = await request.get('https://api.example.com/users');
  expect(response.status()).toBe(200);
  const users = await response.json();
  expect(users[0]).toHaveProperty('email');
});

// POST
test('POST create user', async ({ request }) => {
  const response = await request.post('https://api.example.com/users', {
    data: { name: 'John Doe', email: 'john@example.com', role: 'user' },
  });
  expect(response.status()).toBe(201);
});

// Authenticated request
await request.get('/api/admin/users', { headers: { Authorization: 'Bearer <token>' } });
```

GraphQL:

```
const query = `query GetUser($id: ID!) { user(id: $id) { id name posts { title } } }`;
const response = await request.post('/graphql', { data: { query, variables: { id: '123' } } });
const result = await response.json();
expect(result.data.user.posts).toHaveLength(5);
```

Data setup/teardown via API hooks:

```
test.beforeEach(async ({ request }) => {
  await request.post('/api/products', { data: [{ name: 'Laptop', price: 999 }] });
});
test.afterEach(async ({ request }) => {
  await request.delete('/api/test-data');
});
```

20. Visual Regression Testing

```
await expect(page).toHaveScreenshot('homepage.png');
await expect(page.locator('header')).toHaveScreenshot('header.png');

await expect(page).toHaveScreenshot('dashboard.png', {
  maxDiffPixels: 100,
  threshold: 0.2,
  mask: [page.locator('.timestamp'), page.locator('.balance')], // hide dynamic content
  animations: 'disabled',
});
```

Responsive sweep:

```
const viewports = [
  { name: 'mobile', width: 375, height: 667 },
  { name: 'tablet', width: 768, height: 1024 },
  { name: 'desktop', width: 1920, height: 1080 },
];
for (const vp of viewports) {
  test(`visual - ${vp.name}`, async ({ page }) => {
    await page.setViewportSize(vp);
    await page.goto('/');
    await expect(page).toHaveScreenshot(`page-${vp.name}.png`);
  });
}
```

Update baselines: `npx playwright test --update-snapshots`

21. Accessibility Testing

```
npm install -D @axe-core/playwright
```

```
import AxeBuilder from '@axe-core/playwright';

test('homepage accessibility', async ({ page }) => {
  await page.goto('/');
  const results = await new AxeBuilder({ page })
    .withTags(['wcag2a', 'wcag2aa', 'wcag21aa'])
    .exclude('.third-party-widget')
    .analyze();
  expect(results.violations).toEqual([]);
});
```

 **Domain note:** Regulated industries (e.g., banking) must meet ADA/WCAG 2.1 AA standards; failing audits can carry legal and financial consequences. Key criteria to track: 1.1.1 Text Alternatives, 1.3.1 Info & Relationships, 1.4.3 Contrast, 2.1.1 Keyboard, 2.4.7 Focus Visible, 3.3.2 Labels/Instructions.

22. Performance Testing

```
test('Core Web Vitals', async ({ page }) => {
  await page.goto('/');
  const metrics = await page.evaluate(() => {
    const nav = performance.getEntriesByType('navigation')[0] as PerformanceNavigationTiming;
    return {
      LCP: performance.getEntriesByType('largest-contentful-paint')[0]?.startTime,
      domContentLoaded: nav.domContentLoadedEventEnd - nav.domContentLoadedEventStart,
    };
  });
  expect(metrics.LCP).toBeLessThan(2500);
});
```

Performance budget pattern:

```
const budget = { pageLoadTime: 3000, totalPageSize: 2 * 1024 * 1024, numberOfRequests: 50 };
expect(metrics.pageLoadTime).toBeLessThan(budget.pageLoadTime);
```

API latency:

```
test('API response time', async ({ request }) => {
  const start = Date.now();
  const response = await request.get('/api/users');
  expect(Date.now() - start).toBeLessThan(500);
});
```

23. Component Testing (Playwright CT)

Not covered by either source document — this is a distinct mode from E2E testing that renders individual UI components (React/Vue/Svelte) in a real browser without a full app.

```
npm init playwright@latest -- --ct
```

```
// button.spec.tsx
import { test, expect } from '@playwright/experimental-ct-react';
import { Button } from './Button';

test('button renders and handles click', async ({ mount }) => {
  let clicked = false;
  const component = await mount(<Button onClick={() => (clicked = true)}>Submit</Button>);
  await expect(component).toContainText('Submit');
  await component.click();
  expect(clicked).toBe(true);
});
```

When to reach for it: testing a component's states/props/interactions in isolation (fast, no server/backend needed) — complements, rather than replaces, E2E and integration tests in the Testing Trophy model from Section 1.

PART V — ENTERPRISE ENGINEERING PRACTICES

24. Test Organization & Structure

```
playwright-project/
├─ tests/
│   └─ e2e/{auth,transactions,admin}/
│   └─ api/
│   └─ visual/
│   └─ performance/
├─ pages/           # Page Objects
├─ components/      # Component Objects
├─ fixtures/        # Custom fixtures
├─ helpers/         # Utility functions
├─ data/            # Test data
├─ config/          # Environment configs
└─ playwright.config.ts
```

Best practice: Organize by feature/module, not by test type — keep related tests together for easier maintenance.

25. Custom Fixtures

```
import { test as base } from '@playwright/test';
import { LoginPage } from '../pages/LoginPage';

type CustomFixtures = { loginPage: LoginPage; authenticatedPage: Page; apiClient: any };

export const test = base.extend<CustomFixtures>({
  loginPage: async ({ page }, use) => { await use(new LoginPage(page)); },

  authenticatedPage: async ({ page }, use) => {
    await page.goto('/login');
    await page.getByLabel('Email').fill('user@example.com');
    await page.getByLabel('Password').fill('pass123');
    await page.getByRole('button', { name: 'Log in' }).click();
    await page.waitForURL('/dashboard');
    await use(page);
  },

  apiClient: async ({}, use) => {
    const client = {
      get: (url: string) => fetch(url).then(r => r.json()),
      post: (url: string, data: any) =>
        fetch(url, { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(data) }).then(r =>
          r.json()),
    };
    await use(client);
  },
});
```

26. Data-Driven & Tagged Tests

```
// Tagging
test('critical flow @smoke @critical', async ({ page }) => { /* ... */ });
// npx playwright test --grep @smoke
// npx playwright test --grep-invert @admin

// Data-driven loop
const users = [
  { role: 'admin', email: 'admin@example.com', expectedUrl: '/admin' },
  { role: 'user', email: 'user@example.com', expectedUrl: '/dashboard' },
];
for (const u of users) {
  test(`${u.role} login flow`, async ({ page }) => {
    await page.goto('/login');
    await page.getByLabel('Email').fill(u.email);
    await page.getByRole('button', { name: 'Log in' }).click();
    await expect(page).toHaveURL(u.expectedUrl);
  });
}

// Serial/dependent tests
test.describe.serial('checkout flow', () => {
  test('add to cart', async ({ page }) => {});
  test('proceed to checkout', async ({ page }) => {});
});
```

27. CI/CD Integration & Test Sharding

```
# .github/workflows/playwright.yml
name: Playwright Tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
    matrix:
      shardIndex: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
      shardTotal: [10]
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-node@v3
        with: { node-version: 18 }
      - run: npm ci
      - run: npx playwright install --with-deps
      - run: npx playwright test --shard=${{ matrix.shardIndex }}/${{ matrix.shardTotal }}
      - uses: actions/upload-artifact@v3
        if: always()
        with: { name: playwright-report-${{ matrix.shardIndex }}, path: playwright-report/ }
```

The sharding math: 1,000 tests × 5s each = ~83 minutes serial. With 10 shards → ~8.3 minutes. With 20 shards → ~4.2 minutes.

```
// playwright.config.ts
workers: process.env.CI ? 2 : undefined,
fullyParallel: true,
retries: process.env.CI ? 2 : 0,
```

Docker:

```
FROM mcr.microsoft.com/playwright:v1.40.0-focal
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
CMD ["npx", "playwright", "test"]
```

Best practice: 10 shards covers most projects; use 20+ for suites of 2,000+ tests. GitHub Actions supports up to 256 parallel jobs.

28. Test Reporting (HTML, JUnit, Allure)

Neither source document goes beyond `reporter: 'html'` — here's the full picture for CI integration:

```
// playwright.config.ts — multiple reporters at once
reporter: [
  ['html', { open: 'never' }],
  ['junit', { outputFile: 'results.xml' }], // for Jenkins/Azure DevOps/GitLab
  ['json', { outputFile: 'results.json' }],
  ['github'], // annotates PRs in GitHub Actions
],
```

```
# Allure (popular for enterprise dashboards, trend history, flaky-test tracking)
npm install -D allure-playwright
npx playwright test --reporter=line,allure-playwright
npx allure generate ./allure-results --clean
npx allure open
```

```
npx playwright show-report # view the built-in HTML report locally
```

29. Debugging & Troubleshooting

```
npx playwright test --debug
npx playwright test login.spec.ts --debug
npx playwright test --headed --slow-mo=1000
```

```
await page.pause(); // opens the Inspector mid-test

// Network/console diagnostics
page.on('request', r => console.log('→', r.method(), r.url()));
page.on('requestfailed', r => console.log('x', r.url(), r.failure()?.errorText));
page.on('console', msg => console.log('Browser log:', msg.text()));
page.on('pageerror', err => console.log('Page error:', err.message));

// Element inspection
console.log(`Found ${await button.count()} elements`);
await button.highlight();
console.log('Position:', await button.boundingBox());
```

Trace Viewer (screenshots, DOM snapshots, network, console — replay a failure without rerunning it):

```
use: { trace: 'on-first-retry', screenshot: 'only-on-failure', video: 'retain-on-failure' }
```

```
npx playwright show-trace trace.zip
```

Common fixes:

```
test.setTimeout(120000); // extend a slow test
await page.getByRole('button').click({ timeout: 60000 }); // extend one action

// ✖ Bad
await page.waitForTimeout(3000);
// ✔ Good
await page.waitForLoadState('networkidle');
```

30. Advanced Resilience & Self-Healing Patterns

Resilience testing — see Section 15 (503 simulation, slow-network simulation).

Global teardown for database/test-data cleanup:

```
// global-teardown.ts
async function globalTeardown() {
  await fetch('http://localhost:8000/api/cleanup', { method: 'DELETE', headers: { 'X-Test-Cleanup': 'true' } });
}
export default globalTeardown;
```

Self-healing locators — fall back through alternate selectors when the primary one breaks:

```
export async function smartLocator(page: Page, primary: string, fallbacks: string[]) {
  let element = page.locator(primary);
  if (await element.count() > 0) return element;
  for (const selector of fallbacks) {
    element = page.locator(selector);
    if (await element.count() > 0) return element;
  }
  throw new Error(`Could not locate element: ${primary}`);
}
```

Best practice: Self-healing fallback chains can cut selector-related maintenance by roughly 60% — tests degrade gracefully instead of failing outright when a class name changes.

31. Codegen & the Trace Recorder

Not mentioned in either source, but usually the very first tool a new Playwright user should learn — it records live browser interaction and generates the corresponding test code.

```
npx playwright codegen https://example.com
npx playwright codegen --target=typescript --output=tests/generated.spec.ts https://example.com
npx playwright codegen --device="iPhone 14" https://example.com # record on a mobile emulation profile
```

Use it to scaffold a first draft, then refactor generated CSS-based selectors into role/label-based locators per Section 5, and extract shared logic into Page/Component Objects per Section 9.

32. Environment & Secrets Management

Neither document addresses how to avoid hardcoding credentials/base URLs — a near-guaranteed interview topic.

```
// playwright.config.ts
use: { baseURL: process.env.BASE_URL ?? 'http://localhost:3000' },
```

```
# .env (loaded via dotenv, gitignored)
BASE_URL=https://staging.example.com
TEST_USER_EMAIL=test@example.com
TEST_USER_PASSWORD=***
```

```
import 'dotenv/config';
await page.getByLabel('Email').fill(process.env.TEST_USER_EMAIL!);
```

In CI, inject the same variables as encrypted secrets (GitHub Actions `secrets.*`, Azure DevOps variable groups, etc.) rather than committing a `.env` file.

PART VI — THE FUTURE

33. AI-Driven Testing

Generating tests from natural language — LLMs can draft boilerplate from a plain-English spec (“navigate to /login, enter account + PIN, handle OTP, verify dashboard”), which you then refine for edge cases and business logic.

AI-assisted self-healing — beyond static fallback chains (Section 30), an AI layer can inspect live page HTML and infer the correct selector when all fallbacks fail:

```
async function findElementWithAI(page: Page, context: string) {
  const html = await page.content();
  const response = await openai.chat.completions.create({
    model: 'gpt-4',
    messages: [{ role: 'user', content: `Find the best selector for: ${context}\n\nHTML:\n${html}` }],
  });
  return response.choices[0].message.content ?? '';
}
```

AI-generated edge-case data — for fields that need broad fuzz coverage (special characters, unicode, injection attempts, empty/whitespace/null-byte inputs), an LLM can generate large batches of edge cases far faster than writing them by hand.

AI-assisted maintenance — analyzing historical run data to flag flaky tests (e.g., a test passing 95/100 runs) and suggest concrete fixes (explicit waits, longer timeouts, retry logic), or reviewing code diffs to propose new test cases.

Where this is heading: AI won't replace test engineers — it shifts them from writing boilerplate scripts toward test strategy, domain modeling, and reviewing AI-proposed coverage. The valuable skills become prompt engineering, AI-tool integration, and deep Playwright architecture knowledge, not raw script-writing speed.

PART VII — INTERVIEW PREPARATION

34. Scenario-Based Questions (Deep Dives)

These scenarios are genuinely distinct from the topic sections above and are kept in full — they’re the kind of “walk me through how you’d test X” questions that come up in senior SDET interviews.

Q: How would you test a dynamic table with infinite scrolling?

```
test('infinite scroll with network wait', async ({ page }) => {
  await page.goto('/users');
  for (let i = 0; i < 5; i++) {
    const responsePromise = page.waitForResponse('**/api/users*');
    await page.evaluate(() => window.scrollTo(0, document.body.scrollHeight));
    await responsePromise; // wait for the real trigger, not a fixed timeout
  }
});
```

Preferred approach: track row count and scroll until it stops increasing, or — better — wait explicitly for the API call the scroll triggers, rather than a fixed `waitForTimeout`.

Q: How would you test a multi-step form with validation and back/forward navigation?

```
test('multi-step form submission', async ({ page }) => {
  await page.goto('/signup');
  await page.getByLabel('First Name').fill('John');
  await page.getByLabel('Email').fill('john@example.com');
  await page.getByRole('button', { name: 'Next' }).click();

  // Back navigation should preserve entered data
  await page.getByRole('button', { name: 'Back' }).click();
  await expect(page.getByLabel('First Name')).toHaveValue('John');

  await page.getByRole('button', { name: 'Next' }).click();
  const responsePromise = page.waitForResponse('**/api/signup');
  await page.getByRole('button', { name: 'Submit' }).click();
  expect((await responsePromise).status()).toBe(201);
});
```

Q: How would you test a drag-and-drop Kanban board?

```
test('drag and drop task between columns', async ({ page }) => {
  const todoColumn = page.locator('[data-column="todo"]');
  const inProgressColumn = page.locator('[data-column="in-progress"]');
  const task = todoColumn.locator('[data-task="task-1"]');

  await task.dragTo(inProgressColumn);

  await expect(todoColumn.locator('[data-task="task-1"]')).toHaveCount(0);
  await expect(inProgressColumn.locator('[data-task="task-1"]')).toBeVisible();

  const response = await page.waitForResponse('**/api/tasks/*/move');
  expect(response.status()).toBe(200);
});
```

Fallback for components that don’t support native `dragTo` (custom drag libraries): drive raw `page.mouse.move/down/up` coordinates using each element’s `boundingBox()`.

Q: How would you test a real-time chat application?

```

test('real-time chat messaging', async ({ browser }) => {
  const [context1, context2] = await Promise.all([browser.newContext(), browser.newContext()]);
  const [page1, page2] = await Promise.all([context1.newPage(), context2.newPage()]);

  await page1.goto('/chat'); await page1.fill('#username', 'Alice'); await page1.click('#join');
  await page2.goto('/chat'); await page2.fill('#username', 'Bob'); await page2.click('#join');

  await page1.fill('#message-input', 'Hello Bob!');
  await page1.click('#send');

  await expect(page2.locator('.message').last()).toContainText('Hello Bob!');

  await context1.close(); await context2.close();
});

```

For the underlying transport, Playwright can also observe WebSocket frames directly (`page.on('websocket', ws => ws.on('framereceived', ...))`) and simulate disconnects with `context.setOffline(true)` .

35. Best Practices & Common Pitfalls Cheat Sheet

Do: - Prefer `getByRole()` / `getByLabel()` / `getByText()` over CSS/XPath - Trust auto-waiting; reserve explicit waits for genuinely async, non-DOM conditions - Use Page Objects + Component Objects to isolate change - Parallelize (`fullyParallel: true`) and shard large suites in CI - Use `expect()` (auto-retrying) rather than manually reading + comparing values - Wait for specific network responses instead of the page just “looking” loaded - Give tests descriptive names describing the user behavior under test - Clean up test data in `afterEach` / global teardown - Turn on `trace: 'on-first-retry'` and `screenshot: 'only-on-failure'` for CI debugging - Keep tests independent — no test should depend on another test’s side effects (unless deliberately using `test.describe.serial`)

Avoid: - Fixed sleeps (`waitForTimeout`) as a substitute for a real wait condition - `page.$()` / `page.$$()` `ElementHandle` s where a `Locator` would auto-wait and re-resolve - Brittle selectors tied to DOM structure or class names that change often - Ignoring network state — race conditions between UI assertions and pending API calls are a top cause of flakiness - Hardcoded credentials/URLs — use environment configuration (Section 32) - Over-mocking to the point you’re no longer testing real integration behavior - Skipping accessibility — role-based locators double as an accessibility check for free

End of consolidated guide. Sections marked  (16–18, 23, 28, 31–32) were not present in either source document and have been added to round out common real-world and interview-relevant gaps: storage/cookies, Shadow DOM, mobile emulation, component testing, reporting, codegen, and secrets management.